



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

**SECP3623
Database Programming
2025/2026 SEMESTER 1
Assignment 1 (10%)**

Name	:	LAM YOKE YU
Matric Id	:	A23CS0233
Section	:	02

Name	:	TAN YI YA
Matric Id	:	A23CS0187
Section	:	02

Task 1: Database Creation and Integrity Constraints

1. Create database

```
8 • CREATE DATABASE host_mgmt_yokeyu_yiya;  
9 • USE host_mgmt_yokeyu_yiya;
```

2. Create tables with entity and referential integrity

3. Apply constraints

```
13 • CREATE TABLE room_types(  
14     type_id INT PRIMARY KEY,  
15     type_name VARCHAR(10) NOT NULL,  
16     rent DOUBLE NOT NULL,  
17     deposit DOUBLE NOT NULL,  
18     capacity INT NOT NULL  
19 );  
20  
21 • CREATE TABLE rooms(  
22     room_id INT PRIMARY KEY,  
23     room_no VARCHAR(5) NOT NULL,  
24     floor_no INT NOT NULL,  
25     is_occupied BOOL DEFAULT FALSE,  
26     type_id INT NOT NULL,  
27     FOREIGN KEY (type_id) REFERENCES room_types(type_id)  
28 );  
  
30 • CREATE TABLE students(  
31     student_id INT PRIMARY KEY,  
32     fname VARCHAR(20) NOT NULL,  
33     lname VARCHAR(20) NOT NULL,  
34     status VARCHAR(20) NOT NULL,  
35     checkin_date DATE,  
36     room_id INT,  
37     FOREIGN KEY (room_id) REFERENCES rooms(room_id)  
38 );  
39  
40 • CREATE TABLE maintenance(  
41     maint_id INT PRIMARY KEY,  
42     room_id INT NOT NULL,  
43     issue_desc TEXT NOT NULL,  
44     severity VARCHAR(10),  
45     status VARCHAR(10) NOT NULL,  
46     reported_on DATE NOT NULL,  
47     resolved_on DATE,  
48     FOREIGN KEY (room_id) REFERENCES rooms(room_id)  
49 );
```

```
51 • CREATE TABLE payments(  
52     payment_id INT PRIMARY KEY,  
53     student_id INT NOT NULL,  
54     amount DOUBLE NOT NULL,  
55     paid_on DATE NOT NULL,  
56     method VARCHAR(5) NOT NULL,  
57     note TEXT,  
58     FOREIGN KEY (student_id) REFERENCES students(student_id)  
59 );
```

5. Schema maintenance

5.1 Alter table

```
63 • ALTER TABLE students  
64     ADD COLUMN email VARCHAR(30) UNIQUE;
```

5.2 Drop table

```
67 • CREATE TABLE testing(  
68     test_id INT PRIMARY KEY,  
69     test_desc TEXT NOT NULL  
70 );  
71  
72 • DROP TABLE testing;
```

Task 2: Data Manipulation and Filtering

1. Data Insertion (DML):

The screenshot shows a database IDE interface. The top toolbar includes icons for file operations, a search icon, and a 'Limit to 2000 rows' dropdown. The SQL editor contains the following code:

```
76 • INSERT INTO room_types (type_id, type_name, rent, deposit, capacity) VALUES
77     (1, 'Single', 550, 300, 1),
78     (2, 'Double', 700, 350, 2),
79     (3, 'Premium', 900, 450, 2),
80     (4, 'Family', 1200, 600, 4),
81     (5, 'Economy', 450, 250, 1),
82     (6, 'Twin', 750, 350, 2),
83     (7, 'Suite', 1000, 500, 3),
84     (8, 'Deluxe', 850, 400, 2),
85     (9, 'Dorm', 400, 200, 6),
86     (10, 'Studio', 650, 300, 2);
87 • SELECT * FROM room_types;
```

Below the editor is the 'Result Grid' tab, which displays the data inserted into the `room_types` table. The grid has columns: `type_id`, `type_name`, `rent`, `deposit`, and `capacity`. The data is as follows:

type_id	type_name	rent	deposit	capacity
1	Single	550	300	1
2	Double	700	350	2
3	Premium	900	450	2
4	Family	1200	600	4
5	Economy	450	250	1
6	Twin	750	350	2
7	Suite	1000	500	3
8	Deluxe	850	400	2
9	Dorm	400	200	6
10	Studio	650	300	2
* NULL	NULL	NULL	NULL	NULL

At the bottom, the tab is labeled 'room_types 2' with 'Apply' and 'Revert' buttons.

The screenshot shows the same database IDE interface. The SQL editor contains the following code:

```
89 • INSERT INTO rooms (room_id, room_no, floor_no, is_occupied, type_id) VALUES
90     (1, 'A101', 1, TRUE, 2),
91     (2, 'A102', 1, FALSE, 2),
92     (3, 'A103', 1, TRUE, 2),
93     (4, 'A201', 2, FALSE, 1),
94     (5, 'A202', 2, TRUE, 1),
95     (6, 'A301', 1, TRUE, 5),
96     (7, 'B101', 1, FALSE, 4),
97     (8, 'B201', 2, TRUE, 8),
98     (9, 'C101', 1, FALSE, 9),
99     (10, 'C201', 2, TRUE, 10);
100 • SELECT * FROM rooms;
```

Below the editor is the 'Result Grid' tab, which displays the data inserted into the `rooms` table. The grid has columns: `room_id`, `room_no`, `floor_no`, `is_occupied`, and `type_id`. The data is as follows:

room_id	room_no	floor_no	is_occupied	type_id
1	A101	1	1	2
2	A102	1	0	2
3	A103	1	1	2
4	A201	2	0	1
5	A202	2	1	1
6	A301	1	1	5
7	B101	1	0	4
8	B201	2	1	8
9	C101	1	0	9
10	C201	2	1	10
* NULL	NULL	NULL	NULL	NULL

At the bottom, the tab is labeled 'rooms 3' with 'Apply' and 'Revert' buttons.

Assignment 1 _ Lam Yoke Yu

Limit to 2000 rows

```

102 • INSERT INTO students (student_id, fname, lname, status, checkin_date, room_id, email) VALUES
103     (1, 'Aiman', 'Rahman', 'Active', '2025-01-10', 1, 'aiman@graduate.utm.my'),
104     (2, 'Siti', 'Zahra', 'Active', '2025-01-12', 3, 'siti@graduate.utm.my'),
105     (3, 'Daniel', 'Tan', 'Active', '2025-02-01', 5, 'daniel@graduate.utm.my'),
106     (4, 'Nurul', 'Izzah', 'Inactive', NULL, NULL, 'nurul@graduate.utm.my'),
107     (5, 'Arif', 'Hakim', 'Active', '2025-01-15', 6, 'arif@graduate.utm.my'),
108     (6, 'Li', 'Wei', 'Active', '2025-02-05', 8, 'li@graduate.utm.my'),
109     (7, 'Amira', 'Sofea', 'Active', '2025-03-01', 10, 'amira@graduate.utm.my'),
110     (8, 'Ravi', 'Kumar', 'Inactive', NULL, NULL, 'ravi@graduate.utm.my'),
111     (9, 'Farah', 'Nadia', 'Active', '2025-02-20', 7, 'farah@graduate.utm.my'),
112     (10, 'Hafiz', 'Azlan', 'Active', '2025-03-10', 9, 'hafiz@graduate.utm.my');
113 • SELECT * FROM students;
114

```

Result Grid

	student_id	fname	lname	status	checkin_date	room_id	email
▶	1	Aiman	Rahman	Active	2025-01-10	1	aiman@graduate.utm.my
	2	Siti	Zahra	Active	2025-01-12	3	siti@graduate.utm.my
	3	Daniel	Tan	Active	2025-02-01	5	daniel@graduate.utm.my
	4	Nurul	Izzah	Inactive	NULL	NULL	nurul@graduate.utm.my
	5	Arif	Hakim	Active	2025-01-15	6	arif@graduate.utm.my
	6	Li	Wei	Active	2025-02-05	8	li@graduate.utm.my
	7	Amira	Sofea	Active	2025-03-01	10	amira@graduate.utm.my
	8	Ravi	Kumar	Inactive	NULL	NULL	ravi@graduate.utm.my
	9	Farah	Nadia	Active	2025-02-20	7	farah@graduate.utm.my
	10	Hafiz	Azlan	Active	2025-03-10	9	hafiz@graduate.utm.my

students 6 x

Apply Revert

Assignment 1 _ Lam Yoke Yu

Limit to 2000 rows

```

115 • INSERT INTO maintenance (maint_id, room_id, issue_desc, severity, status, reported_on, resolved_on) VALUES
116     (1, 7, 'Water leakage', 'HIGH', 'RESOLVED', '2025-01-12', '2025-01-15'),
117     (2, 3, 'Broken chair', 'LOW', 'RESOLVED', '2025-02-03', '2025-02-05'),
118     (3, 6, 'Fan have noise', 'MEDIUM', 'RESOLVED', '2025-03-22', '2025-03-24'),
119     (4, 2, 'Door lock broken', 'HIGH', 'RESOLVED', '2025-03-25', '2025-03-26'),
120     (5, 9, 'Missing pillow', 'LOW', 'OPEN', '2025-04-04', NULL),
121     (6, 4, 'Broken window', 'HIGH', 'RESOLVED', '2025-04-15', '2025-04-17'),
122     (7, 1, 'Unstable Wifi connection', 'LOW', 'OPEN', '2025-05-02', NULL),
123     (8, 10, 'Power socket not functioning', 'MEDIUM', 'RESOLVED', '2025-07-12', '2025-07-13'),
124     (9, 5, 'Window lock rusty', 'LOW', 'OPEN', '2025-08-05', NULL),
125     (10, 8, 'Air conditioner not working', 'MEDIUM', 'OPEN', '2025-11-08', NULL);
126 • SELECT * FROM maintenance;
127

```

Result Grid

	maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on
▶	1	7	Water leakage	HIGH	RESOLVED	2025-01-12	2025-01-15
	2	3	Broken chair	LOW	RESOLVED	2025-02-03	2025-02-05
	3	6	Fan have noise	MEDIUM	RESOLVED	2025-03-22	2025-03-24
	4	2	Door lock broken	HIGH	RESOLVED	2025-03-25	2025-03-26
	5	9	Missing pillow	LOW	OPEN	2025-04-04	NULL
	6	4	Broken window	HIGH	RESOLVED	2025-04-15	2025-04-17
	7	1	Unstable Wifi connection	LOW	OPEN	2025-05-02	NULL
	8	10	Power socket not functioning	MEDIUM	RESOLVED	2025-07-12	2025-07-13
	9	5	Window lock rusty	LOW	OPEN	2025-08-05	NULL
	10	8	Air conditioner not working	MEDIUM	OPEN	2025-11-08	NULL

maintenance 7 x

Apply Revert

Assignment 1 _ Lam Yoke Yu ...

Limit to 2000 rows

```

128 • INSERT INTO payments (payment_id, student_id, amount, paid_on, method, note) VALUES
129 (1, 1, 700.00, '2025-01-10', 'CASH', 'Aiman, A101, January'),
130 (2, 2, 700.00, '2025-01-12', 'FPX', 'Siti, A103, January'),
131 (3, 3, 550.00, '2025-01-15', 'CARD', 'Daniel, A202, January'),
132 (4, 5, 450.00, '2025-01-20', 'TNG', 'Arif, A301, January'),
133 (5, 6, 850.00, '2025-01-25', 'FPX', 'Li, B201, January'),
134 (6, 7, 650.00, '2025-02-01', 'CARD', 'Amira, C201, February'),
135 (7, 9, 1200.00, '2025-02-05', 'CASH', 'Farah, B101, February'),
136 (8, 10, 400.00, '2025-02-07', 'TNG', 'Hafiz, C101, February'),
137 (9, 1, 700.00, '2025-02-10', 'FPX', 'Aiman, A101, February'),
138 (10, 2, 700.00, '2025-02-12', 'CARD', 'Siti, A103, February');
139 • SELECT * FROM payments;
140

```

Result Grid

payment_id	student_id	amount	paid_on	method	note
1	1	700	2025-01-10	CASH	Aiman, A101, January
2	2	700	2025-01-12	FPX	Siti, A103, January
3	3	550	2025-01-15	CARD	Daniel, A202, January
4	5	450	2025-01-20	TNG	Arif, A301, January
5	6	850	2025-01-25	FPX	Li, B201, January
6	7	650	2025-02-01	CARD	Amira, C201, February
7	9	1200	2025-02-05	CASH	Farah, B101, February
8	10	400	2025-02-07	TNG	Hafiz, C101, February
9	1	700	2025-02-10	FPX	Aiman, A101, February
10	2	700	2025-02-12	CARD	Siti, A103, February

payments 8 x

Apply Revert

2. UPDATE and DELETE Operations:

Assignment 1 _ Lam Yoke Yu ...

Limit to 2000 rows

```

143 • SET SQL_SAFE_UPDATES = 0;
144 • UPDATE rooms r
145 LEFT JOIN students s
146 ON r.room_id = s.room_id
147 SET r.is_occupied =
148 CASE
149 WHEN s.status = 'Active' THEN TRUE
150 ELSE FALSE
151 END;
152 • SELECT * FROM rooms;

```

SQLAdditions

Automatic context help disabled. Use the toolbar manually get help for the current caret position or toggle automatic help

Result Grid

room_id	room_no	floor_no	is_occupied	type_id
1	A101	1	1	2
2	A102	1	0	2
3	A103	1	1	2
4	A201	2	0	1
5	A202	2	1	1
6	A301	1	1	5
7	B101	1	1	4
8	B201	2	1	8

rooms 9 x

Apply Revert

Context Help Snippets

Output

#	Time	Action	Message	Duration / Fetch
34	23:18:55	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
35	23:19:03	UPDATE rooms r LEFT JOIN students s ON r.room_id = s.room_id SET r.is_occu...	2 row(s) affected Rows matched: 10 Changed: 2 Warnings: 0	0.000 sec
36	23:19:39	SELECT * FROM rooms LIMIT 0, 2000	10 row(s) returned	0.000 sec / 0.000 sec

First, safe mode is disabled. Then, LEFT JOIN the rooms table with the students table using room_id. is_occupied in rooms table is updated with CASE, which is SET to true when student status is active and vice versa.

Assignment 1 - Lam Yoke Yu

```

153
154 # delete operation
155 • DELETE FROM maintenance
156 WHERE
157     status = 'RESOLVED' AND
158     reported_on <= DATE_SUB(CURDATE(), INTERVAL 60 DAY);
159 • SET SQL_SAFE_UPDATES = 1;
160 • SELECT * FROM maintenance;
161

```

Limit to 2000 rows

	maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on
▶	5	9	Missing pillow	LOW	OPEN	2025-04-04	NULL
	7	1	Unstable WiFi connection	LOW	OPEN	2025-05-02	NULL
	9	5	Window lock rusty	LOW	OPEN	2025-08-05	NULL
	10	8	Air conditioner not working	MEDIUM	OPEN	2025-11-08	NULL
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

maintenance 13 x | Apply | Revert | Context Help | Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
5	23:24:51	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
6	23:25:35	DELETE FROM maintenance WHERE status = 'RESOLVED' AND reported_on ...	6 row(s) affected	0.015 sec
7	23:25:38	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec
8	23:25:40	SELECT * FROM maintenance LIMIT 0, 2000	4 row(s) returned	0.000 sec / 0.000 sec

Using DELETE FROM, the row that has status marked as RESOLVED and is 60 days old will be deleted from the maintenance table. Then, safe mode is enabled for safety.

3. Data Retrieval and Filtering Queries:

```

162 # 3. Data retrieval and filtering queries
163 #Display room types where rent is between rm400 and rm800
164 • SELECT type_name FROM room_types WHERE rent BETWEEN 400 AND 800;
165

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	type_name
▶	Single
	Double
	Economy
	Twin
	Dorm
	Studio

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

```

166 #Retrieve students whose fname start with the letter 'A'
167 • SELECT fname, lname FROM students WHERE fname LIKE 'A%';
168

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	fname	lname
▶	Aiman	Rahman
	Arif	Hakim
	Amira	Sofea

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

```

169 #Display payments where the payment method is FPX or CARD
170 • SELECT * FROM payments WHERE method IN ('FPX', 'CARD');

```

```

171

```

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: [IA](#)

	payment_id	student_id	amount	paid_on	method	note
▶	2	2	700	2025-01-12	FPX	Siti, A103, January
	3	3	550	2025-01-15	CARD	Daniel, A202, January
	5	6	850	2025-01-25	FPX	Li, B201, January
	6	7	650	2025-02-01	CARD	Amira, C201, February
	9	1	700	2025-02-10	FPX	Aiman, A101, February
	10	2	700	2025-02-12	CARD	Siti, A103, February
*	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid

Form Editor

```

171
172 #combine AND, OR and NOT
173 • SELECT m.room_id, m.issue_desc, m.severity, m.status, r.room_no, r.floor_no
174 FROM maintenance m
175 LEFT JOIN rooms r ON m.room_id = r.room_id
176 WHERE m.status = 'OPEN'
177 AND m.severity != 'LOW'
178 AND (
179     m.severity = 'HIGH'
180     OR reported_on <= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
181 );
182

```

Result Grid Filter Rows: Export: Wrap Cell Content: [IA](#)

room_id	issue_desc	severity	status	room_no	floor_no
---------	------------	----------	--------	---------	----------

Result Grid

Form Editor

Result 17 x

Read Only

room_id, issue_desc, severity, status, room_no and floor_no are joined from maintenance and rooms using LEFT JOIN, where status is open AND severity is NOT low AND severity is high OR is left unattained for 30 days or above.

4. Functions and Expressions:

```

184 # Aggregate Function
185 • SELECT AVG(rent) AS average_rent
186 FROM room_types;
187

```

Result Grid Filter Rows: Export: Wrap Cell Content: [IA](#)

average_rent
▶ 745

Result Grid

Form Editor


```
188 # String Functions
189 • SELECT UPPER(CONCAT(fname, ' ', lname)) AS full_name_uppercase, status
190 FROM students;
191
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: I A

	full_name_uppercase	status
▶	AIMAN RAHMAN	Active
	SITI ZAHRA	Active
	DANIEL TAN	Active
	NURUL IZZAH	Inactive
	ARIF HAKIM	Active
	LI WEI	Active
	AMIRA SOFEA	Active
	RAVI KUMAR	Inactive
	FARAH NADIA	Active
	HAFIZ AZLAN	Active

Result Grid

Form Editor

Field Types

Result 19 x

Read Only

Task 3: Reporting and Aggregation

1. Create v_room_status

```
191 # Task 3
192 # 1. Create v_room_status
193 • CREATE VIEW v_room_status AS
194     SELECT r.room_no, rt.type_name, rt.rent, r.floor_no, rt.capacity,
195           COUNT(CASE WHEN s.status = 'ACTIVE' THEN 1 END) AS n_occupants,
196           COUNT(CASE WHEN m.status = 'OPEN' THEN 1 END) AS pending_issues,
197           CASE
198             WHEN COUNT(CASE WHEN s.status = 'ACTIVE' THEN 1 END) = 0 THEN TRUE
199             ELSE FALSE
200           END AS is_vacant
201     FROM rooms r
202     LEFT OUTER JOIN room_types rt ON (r.type_id = rt.type_id)
203     LEFT OUTER JOIN students s ON (r.room_id = s.room_id)
204     LEFT OUTER JOIN maintenance m ON (r.room_id = m.room_id)
205     GROUP BY r.room_no, rt.type_name, rt.rent, r.floor_no, rt.capacity;
206
207 • SELECT * FROM v_room_status;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	room_no	type_name	rent	floor_no	capacity	n_occupants	pending_issues	is_vacant
	A101	Double	700	1	2	1	1	0
	A102	Double	700	1	2	0	0	1
	A103	Double	700	1	2	1	0	0
	A201	Single	550	2	1	0	0	1
	A202	Single	550	2	1	1	1	0
	A301	Economy	450	1	1	1	0	0
	B101	Family	1200	1	4	1	0	0
	B201	Deluxe	850	2	2	1	1	0
	C101	Dorm	400	1	6	1	1	0
	C201	Studio	650	2	2	1	0	0

2. Summary queries

2.1 Total number of students per room type

```
235 # 2.4 Count of OPEN maintenance issues per floor
236 • SELECT r.floor_no, COUNT(m.maint_id) AS open
237     FROM maintenance m
238     LEFT OUTER JOIN rooms r ON (m.room_id = r.room_id)
239     WHERE m.status = 'OPEN'
240     GROUP BY r.floor_no
241     HAVING COUNT(m.maint_id) > 2;
242
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

floor_no

open

After adding several entries the results are as follow:

```

209 # 2. Summary Queries
210 # 2.1 Total number of students per room type
211 • SELECT rt.type_name, COUNT(CASE WHEN s.status = 'ACTIVE' THEN 1 END) AS n_occupants
212 FROM room_types rt
213 LEFT OUTER JOIN rooms r ON (r.type_id = rt.type_id)
214 LEFT OUTER JOIN students s ON (r.room_id = s.room_id)
215 GROUP BY rt.type_name;
216
217

```

Result Grid		
Filter Rows:		
Export:  Wrap Cell Content: 		
	type_name	n_occupants
▶	Single	1
	Double	2
	Premium	0
	Family	1
	Economy	1
	Twin	0
	Suite	0
	Deluxe	1
	Dorm	1
	Studio	1

2.2 Average rent and total deposit per room type

This statement also used the UPPER() and CASE as required in No. 3

```

217 # 2.2 Average rent and total deposit per room type
218 • SELECT UPPER(rt.type_name),
219         ROUND(AVG(rt.rent), 2) AS avg_rent,
220         ROUND(SUM(rt.deposit), 2) AS total_deposit,
221         CASE
222             WHEN AVG(rt.rent) > 1000 THEN 'High'
223             WHEN AVG(rt.rent) > 500 THEN 'Medium'
224             ELSE 'Low'
225         END AS rent
226 FROM students s
227 LEFT OUTER JOIN rooms r ON (r.room_id = s.room_id)
228 LEFT OUTER JOIN room_types rt ON (r.type_id = rt.type_id)
229 WHERE s.status = 'Active'
230 GROUP BY rt.type_name;
231

```

Result Grid			
Filter Rows:			
Export:  Wrap Cell Content: 			
	UPPER(rt.type_name)	avg_rent	total_deposit
▶	DOUBLE	700	700
	SINGLE	550	300
	ECONOMY	450	250
	DELUXE	850	400
	STUDIO	650	300
	FAMILY	1200	600
	DORM	400	200

2.3 Monthly payment totals grouped by year & month

```
227 # 2.3 Monthly payment totals grouped by year & month
228 • SELECT
229     YEAR(paid_on) AS payment_year,
230     MONTH(paid_on) AS payment_month,
231     ROUND(SUM(amount), 2) AS total_payment
232 FROM payments
233 GROUP BY YEAR(paid_on), MONTH(paid_on);
234
```

Result Grid			
		Filter Rows:	
		Export:	
		Wrap Cell Content:	
	payment_year	payment_month	total_payment
▶	2025	1	3250
	2025	2	3650

MARKING RUBRICS

Task	Criteria	Marks	Total Marks
Task 1 Database Creation & Constraints	Creation of database and all tables with valid keys, datatypes, and constraints (PK, FK, NOT NULL, UNIQUE). Logical order and referential integrity preserved.	3	
	ALTER TABLE (add email UNIQUE) and DROP TABLE demonstration executed successfully and documented.	2	
	Insertion order, valid FK references, and no orphan data.	2	
Task 2 Data Manipulation & Filtering	Five tables populated with ≥10 valid records each; varied and meaningful data UPDATE/DELETE executed correctly with logical results.	1	
	Accurate use of WHERE, BETWEEN, LIKE, IN, AND/OR/NOT queries.	3	
	Correct use of aggregate and string functions (COUNT, CONCAT, and more).	2	
Task 3 Reporting & Aggregation	VIEW creation	2	
	Use of GROUP BY, HAVING, ROUND, and CASE with accurate aggregated results.	3	
	Clear labelling, readable outputs, screenshots show executed results.	2	
Total (20)			